



# HarvestHub

**Marketplace for Local Farm Products**

## Software Requirements Specification Version 1.0

**Project Name:** HarvestHub

**Theme:** eGreen Basket

**Category:** Multi-Platform App Computing

# Table of Contents

|     |                                                           |    |
|-----|-----------------------------------------------------------|----|
| 1.1 | Background and Necessity for the Cross-Platform App ..... | 3  |
| 1.2 | Proposed Solution.....                                    | 4  |
| 1.3 | Purpose of the Document .....                             | 5  |
| 1.4 | Scope of Project.....                                     | 5  |
| 1.5 | Constraints .....                                         | 7  |
| 1.6 | Functional Requirements.....                              | 9  |
| 1.7 | Non-Functional Requirements.....                          | 19 |
| 1.8 | Interface Requirements .....                              | 20 |
| 1.9 | Project Deliverables .....                                | 22 |

## 1.1 Background and Necessity for the Cross-Platform App

Agriculture is one of the most important sectors supporting economies and communities worldwide. Local farmers produce fresh fruits, vegetables, grains, dairy products, and other agricultural goods that are essential for daily life. However, many small-scale farmers face challenges in reaching customers directly due to limited digital presence, dependence on intermediaries, and restricted market access.

On the other hand, consumers increasingly prefer purchasing fresh, locally sourced products due to its quality, affordability, and nutritional value. Unfortunately, finding trusted local farmers often requires visiting multiple markets or relying on word-of-mouth recommendations, making the buying process time-consuming and inconvenient.

With the rapid growth of smartphones and mobile apps, digital marketplaces have become an effective way to connect customers and farmers. **HarvestHub** is designed as a lightweight cross-platform app that enables customers to conveniently browse products, discover fresh products, compare prices, place orders, and discover local farmers, while allowing farmers to showcase their products to a larger audience.

The project incorporates three different user roles, namely, Customer, Farmer, and Administrator, each with dedicated functionalities and responsibilities.

## 1.2 Proposed Solution

**HarvestHub** is a cross-platform mobile app that provides a digital marketplace for buying and selling fresh agricultural products.

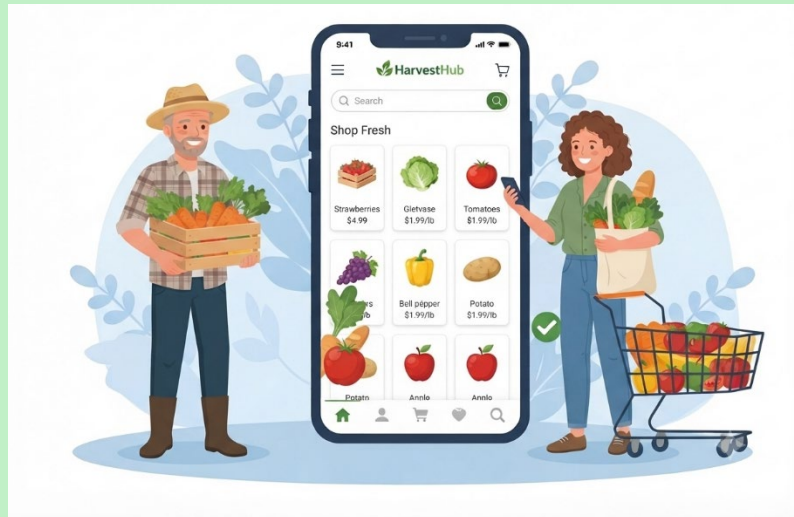
The app offers separate interfaces for customers, farmers, and administrators enabling each user type to perform tasks relevant to their role.

The app allows customers to browse products, search by category, maintain wishlists, manage shopping carts, and place simulated orders. Farmers can manage their products, update inventory, and monitor customer orders through a dedicated farmer dashboard. Administrators oversee the overall operation of the app by managing customers, farmers, products, categories, and orders.

To provide a modern user experience, the app also includes a simple AI Farm Product Assistant (Chatbot) that helps users obtain basic information about fruits, vegetables, nutrition, and storage tips through predefined responses or a basic AI API integration depending on the implementation approach.

### Proposed Technology Stack

| Component                       | Technology                                             |
|---------------------------------|--------------------------------------------------------|
| Mobile App Frameworks and Tools | Flutter SDK 3.x with Dart 2.x OR Android SDK with Java |
| IDE                             | Android Studio/Visual Studio Code                      |
| Database                        | Cloud Firestore/SQLite/Firebase                        |
| Authentication                  | Firebase Authentication                                |
| Networking                      | HTTP Package                                           |
| AI Integration                  | Basic AI API or Predefined Knowledge Base              |



## 1.3 Purpose of the Document

This document presents a comprehensive Software Requirements Specification (SRS) for the **HarvestHub** cross-platform app. It provides clear guidelines for developers, designers, testers, and project managers during the design, development, and testing phases.

## 1.4 Scope of Project

**HarvestHub** provides a digital platform that connects customers with local farmers through a cross-platform app.

The app enables customers to perform actions such as:

- Create and manage user accounts
- Browse products by category
- Search for products using keywords
- View product details
- Add products to a wishlist
- Manage a shopping cart
- Place simulated orders
- View order history
- Use an AI Farm Products Assistant for basic farming and nutrition-related queries

The app enables farmers to perform actions such as:

- Register and manage farmer profiles
- Add, update, and remove products
- Maintain product inventory
- View customer orders
- Update order status (where status can be Pending, Confirmed, Ready for Pickup, Completed, or Cancelled)

The app enables administrators to:

- Manage customers
- Manage farmers
- Manage product categories
- Manage products
- View and manage customer orders
- Monitor app content

The app will also have:

**Push Notifications:** Real-time notifications for both farmers and customers to improve communication and engagement.

**Farmer Follow/Favorite:** Customers can follow or favorite specific farmers, enabling personalized restock alerts.

**Inventory Monitoring:** Automated push notifications for customers when items on their wishlist or from followed farmers are restocked.

**Pickup Management:** Functionality should be included for customers to view available pickup slots, select a slot, and modify or cancel bookings.

**Advanced Search and Filter:** Expanded search capabilities allowing users to filter by market name, location, distance, and specific farmers.

## 1.5 Constraints

Following constraints apply to the development of **HarvestHub**:

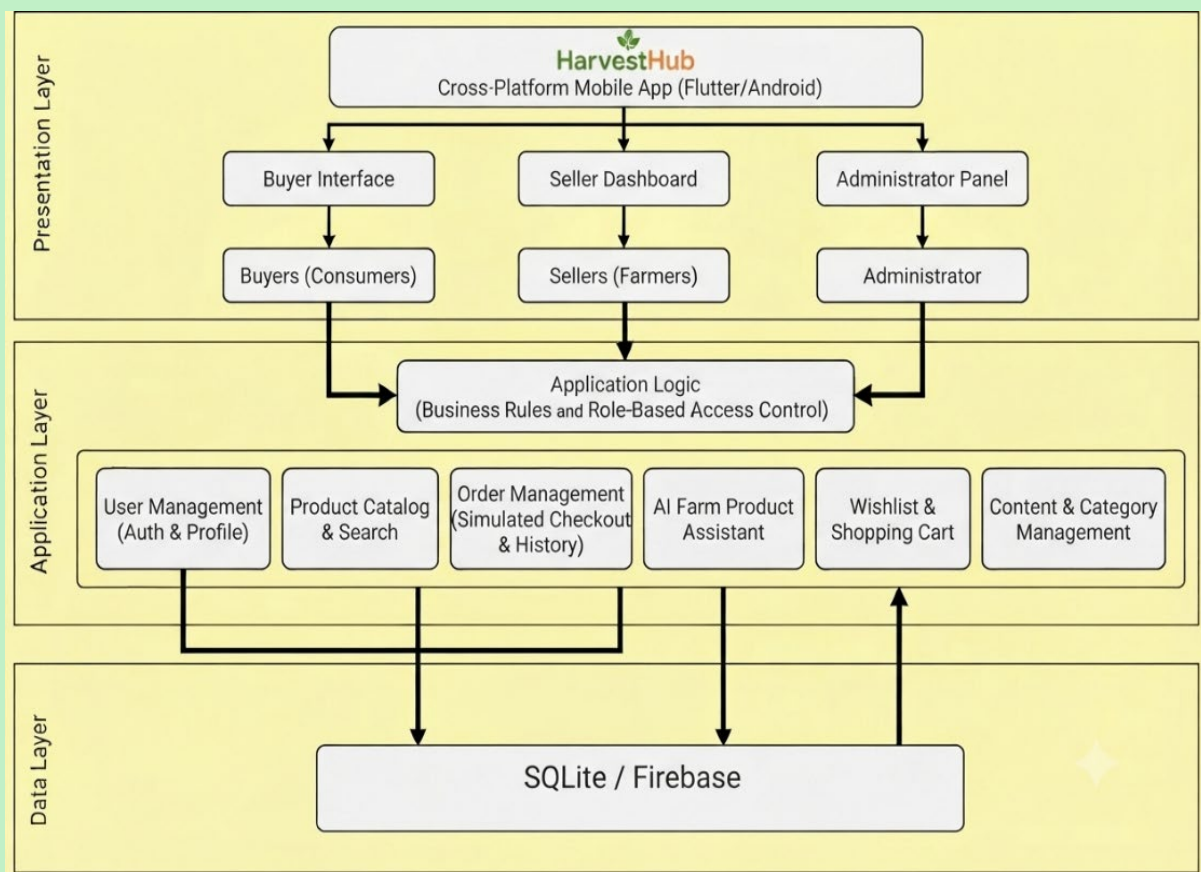
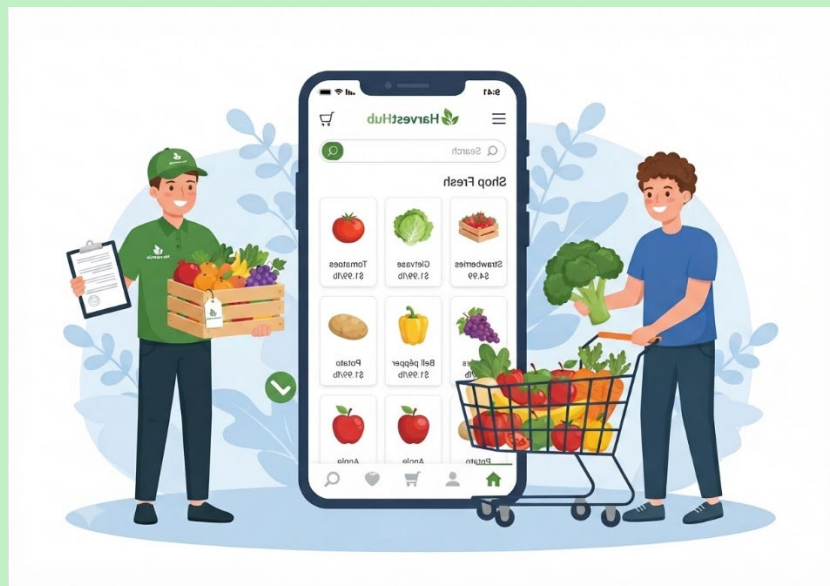
- The cross-platform app must maintain rapid loading times, low memory overhead, and smooth transitions.
- Payment gateway integration is outside the scope of the project.
- Order placement will be simulated and will not involve actual financial transactions.
- Delivery and pickup functionality is not included.
- Push notifications and advanced recommendation systems are outside the scope of the project.
- The AI Farm Products Assistant shall provide basic informational assistance using predefined responses or a simple AI API integration.
- The app shall focus on implementing add/modify/delete operations, authentication, navigation, local/cloud database usage, and role-based access control.

## System Architecture

HarvestHub follows a **three-tier architecture** consisting of the Presentation Layer, App Layer, and Data Layer. This architecture provides a clear separation between the user interface, app logic, and database making the app easier to develop, test, and maintain.

For example, a Flutter mobile app can serve as the presentation layer, providing different interfaces for Customers, Farmers, and Administrators. The app communicates with Firebase Authentication for user authentication, Cloud Firestore (or SQLite) for data storage, and external REST APIs for optional AI assistance.

The app logic processes user requests, validates data, manages product and order information, and coordinates interactions between users and the database.



**System Architecture Diagram**

## 1.6 Functional Requirements

The app will be designed as a set of Forms/Pages, Navigation, and Fragments/Screens with menus representing choice of activities to be performed.

Detailed functional requirements are as follows:

The landing page of the app should accept user role.

### ❖ User Roles

HarvestHub supports **three** different types of users. Each user is provided with a separate interface and specific functionalities based on their responsibilities. Admin does not require sign up - their credentials are preconfigured.

Details of these user roles are as follows:

### ❖ Customer

A Customer is one who uses the app to browse agricultural products, compare available items, manage shopping activities, and place simulated orders.

The Customer can:

- Register and log in
- Maintain a personal profile
- Browse products
- Search products
- View product details
- Add products to a wishlist
- Manage a shopping cart
- Place simulated orders
- View previous orders



- Use the AI Farm Products Assistant

### ❖ **Farmer**

The Farmer can:

- Log in securely
- Manage farmer profile
- Add new products
- Update product details
- Delete products
- Update inventory
- View customer orders
- Update order status
- Prevent products with zero stock to be ordered
- Generate and view reports



### ❖ **Administrator**

The Administrator manages the overall app.

The Administrator can:

- Manage Customers
- Manage Farmers
- Manage Products
- Manage Categories
- Manage Orders
- View platform-wide reports covering total orders, revenue summary across markets, and the most active farmers



Detailed functionalities for each module are elaborated here.

### ❖ **Customer Registration and Login**

The app shall allow new customers to create an account using their basic information.

The registration form shall include: Full Name, Email Address, Mobile Phone Number, Password, and Address.

Registered customers shall be able to log into the app using their email address and password.

The system shall validate login credentials before granting access.

### ❖ **Customer Profile Management**

The app shall allow customers to manage their personal information.

Customers shall be able to:

- View profile
- Edit profile information
- Update contact number
- Update address
- Change password
- Logout

### ❖ **Product Browsing**

The app shall provide a catalog of available agricultural products.

Products may include:

- Fruits
- Vegetables
- Grains
- Dairy Products

- Herbs
- Organic Products



Each product shall display:

- Product Image
- Product Name
- Category
- Price
- Available Quantity
- Farmer Name

### ❖ **Category Management**

The app shall organize products into categories to simplify browsing.

Example categories include:

- Fruits
- Vegetables
- Organic Products
- Dairy
- Pulses
- Herbs and Spices

Customers shall be able to browse products by selecting a category.

### ❖ **Search**

The app shall provide a search facility that enables customers to locate products quickly.

Customers can search and/or filter products by:

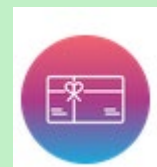
- Product Name
- Category
- Market Name
- Location/Distance
- Farmer Name

Matching products shall be displayed dynamically.

### ❖ **Product Details**

Selecting a product shall display detailed information including:

- Product Image
- Product Name
- Description
- Price
- Available Stock
- Farmer Information
- Product Category



The customer shall be able to add the product to the shopping cart or wishlist from this screen.

### ❖ **Wishlist Management**

The app shall allow customers to save products for future purchase.



Customers shall be able to:

- Add/remove/view products in wishlist

### ❖ **Shopping Cart Management**

The shopping cart shall allow customers to temporarily store products before placing an order.

Customers shall be able to:

- Add/modify/remove products
- Update quantities
- View subtotal
- Clear the cart



### ❖ **Simulated Checkout**

The app shall simulate the order placement process.

During checkout, customers shall be able to:

- Review selected products
- Confirm delivery address
- View total amount
- Confirm order

No actual payment gateway shall be integrated.

Upon successful confirmation, the order shall be recorded in the database.

### ❖ **Order History**

The app shall maintain a history of previously placed orders.

Each order shall display:

- Order ID
- Date
- Purchased Products
- Total Amount
- Current Order Status



### ❖ **AI Farm Products Assistant**

The app must include a simple AI **Farm Products Assistant** to help customers obtain useful information related to agricultural products.



For example, the assistant may answer questions such as:

- Which fruits are rich in Vitamin C?
- How should tomatoes be stored?
- Which vegetables are seasonal?
- What are the health benefits of spinach?
- Suggest vegetables for a healthy salad.



The AI assistant may be implemented using predefined responses or a basic AI API integration based on the project's implementation approach.

Optional support for AI-driven queries regarding farmer availability, product stock, pickup timings, and general market information can also be included in the app.

### ❖ **Farmer Interaction**

Customers can 'follow/favorite' farmers. A button or option can be added to implement this functionality.

### ❖ Restock Alerts

The app shall monitor inventory for wishlisted items and items sold by followed farmers, triggering a push notification to the customer upon restock.

### ❖ Pickup Scheduling

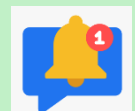
Customers shall view a list of available pickup slots for a selected farmer.

Customers can book a slot during the checkout process.

Customers may modify or cancel existing pickup bookings via the order management section.

### ❖ Order status

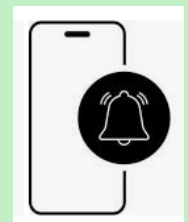
The app shall also send push notifications to customers for order status updates.



### ❖ Push Notifications for Farmers

The system shall send push notifications to farmers for:

- New order notifications.
- Booking cancellations or modifications.



### ❖ Farmer Follow/Favorite

Customers can follow or favorite specific farmers, enabling personalized restock alerts.

### ❖ Report Generation

Farmers can generate and view daily, weekly, or monthly reports.

### ❖ Inventory Monitoring

Automated push notifications for customers when items on their wishlist or from followed farmers are restocked. Push notifications can also be sent to farmers when stock reduces below a specific quantity so that they can restock.

## ❖ Pickup Management

Functionality should be included for customers to view available pickup slots, select a slot, and modify or cancel bookings.

## ❖ About Us

The app shall provide information about **HarvestHub**, its purpose, and its objectives.

## ❖ Contact Us

The app shall provide contact information through which users can reach the app administrators.

Information may include:

- Email Address
- Phone Number
- Office Address
- Feedback Form

## Important Note Regarding AI Usage:

You are encouraged to use AI-powered tools (such as AI-powered UI/UX design tools, code assistants, and image-generation tools) to enhance productivity and creativity. However, AI should be used as a supporting aid rather than a substitute for your own design, development, and problem-solving skills.

Do NOT rely on completely ready-made mobile templates for your project, as this will adversely affect your evaluation. Your Web application's design, structure, and implementation should primarily reflect your own skills and understanding. Do NOT submit AI-generated code or content without meaningful modification and understanding. AI-generated suggestions may be used for guidance, learning, debugging, or improving productivity, but the final solution should demonstrate your own effort, logic, and implementation.

Acknowledge all the AI tool(s) used (for example, Copilot, Canva AI, Figma AI, Uizard, or similar) in your project documentation or submission.

During evaluation, judges may ask participants to explain their design decisions, implementation approach, and code. You are, therefore, expected to understand and be able to justify all aspects of your submitted work.

Do not use AI tools to fully produce ready-made documentation. This is strictly forbidden.

**Bottomline:** AI is your **assistant**, not your **developer**. Your knowledge, creativity, and coding skills should drive the project.

## 1.7 Non-Functional Requirements

- **Performance:** The cross-platform app should demonstrate high value of performance through speed and throughput. In simple terms, the cross-platform app should be fast to load and transitions should be smooth.
- **Security:** The cross-platform app should implement adequate security measures such as authentication. All data must be encrypted; role-based access control implemented.
- **Scalability:** The app architecture and infrastructure should be designed to handle increasing user traffic, data storage, and feature expansions.
- **Accessibility:** The cross-platform app should have clear and legible fonts, user-interface elements, and navigation elements.
- **Reliability:** The cross-platform app should be available 24/7 with minimum downtime.
- **Data Backup:** Scheduled auto-backups for records and logs should be performed.



These are the bare minimum expectations from the project. It is a must to implement the functional and non-functional requirements given in this SRS. Once they are complete, you can use your own creativity and imagination to add more features if required.

## 1.8 Interface Requirements

### Hardware

- Intel Core i7/ i5 Processor or higher
- 8 GB RAM or above
- Color SVGA monitor
- 500 GB Hard Disk space
- Mouse
- Keyboard
- Internet access (4G or Wi-Fi)

### Software

**IDE:** Android Studio IDE with Android 9 or higher with Java OR Flutter 3.24, Dart 3.6 or higher and related libraries

**AI tools:** AI-related Dart packages (such as dart\_ml,string\_similarity and ml\_linalg), TensorFlow Lite, Google ML Kit, Firebase ML Kit, and Google AI Studio for API key creation

**Database:** SQLite or Firebase

### Testing

- You can test the cross-platform app using a mobile device (Android Smartphone or iPhone) or a suitable emulator.

## Database Design

Database may be SQL Lite or Firebase. In Firebase, typically structures are created as collections hence, here are a few sample collections that can be created for this app. Note that the keys are actually created as Document Ids in Firebase. For schema purposes, they are written as PK and FK here.

### Users Collection

|                        |
|------------------------|
| User_Id (PK)           |
| Name                   |
| Email                  |
| Role (Customer/Farmer) |
| Created_At             |

### Farmers Collection

|                |
|----------------|
| Farmer_Id (PK) |
| User_Id (FK)   |
| Market_Id (FK) |
| Business_Name  |
| Description    |
| Rating         |

### Products Collection

|                 |
|-----------------|
| Product_Id (PK) |
| Farmer_Id (FK)  |
| Item_Name       |
| Category        |
| Price_Per_Unit  |
| Stock_Qty       |
| Image_Url       |

### Orders Collection

|                  |
|------------------|
| Order_Id (PK)    |
| Customer_Id (FK) |
| Farmer_Id (FK)   |
| Items_JSON       |
| Pickup_Slot_Time |
| Status           |
| Total_Price      |

### Farmers\_Market Collection

|                 |
|-----------------|
| Market_Id (PK)  |
| Market_Name     |
| Address         |
| GPS_Coordinates |
| Operating_Hours |
| Active_Status   |

## 1.9 Project Deliverables

You must design and build the project and submit it along with a complete project report that includes:

- Problem Definition
- Design Specifications
- Diagrams such as Flowcharts for various Activities, Data Flow Diagrams, and so on
- Database Design
- Test Data Used in the Project
- **Project Installation Instructions (MANDATORY)**
- **User Credentials (Login ID and Password- MANDATORY)**

**Documentation is considered as a very important part of the project.** Ensure that documentation is complete and comprehensive.

**Documentation should not contain any source code.**

The consolidated project will be submitted as a Source Code zip file with a ReadMe.doc file listing assumptions (if any) made at your end. It should SQL Lite and include SQL scripts files (.sql) containing database and table definitions in case of SQL Lite and collection schemas in case of Firebase. Android Package (.apk file) or iOS App Store Package (.ipa file) for the cross-platform app must also be submitted as part of the zip file.

**Submit a video (.mp4 file) demonstrating the working of the cross-platform app, including all the functionalities of the project. This is MANDATORY.**

Over and above the given specifications, you can apply your creativity and logic to improve the cross-platform app.

*~~ End of the Document ~~*